

Converting Maven to Gradle

Converting Maven to Gradle

Converting a Apache Maven project to Gradle means changing the project's build system from Maven to Gradle for better flexibility and faster performance.

Maven uses the pom.xml file to manage project configuration and dependencies, whereas Gradle uses build.gradle files written in Groovy or Kotlin DSL. During conversion, all dependencies, plugins, and build configurations from Maven are transferred into Gradle format.

The main purpose of converting Maven to Gradle is to improve build speed, simplify project configuration, and support modern DevOps automation practices. Gradle provides incremental builds and better customization features, making it more efficient for large-scale projects.

Working Process

1. The existing Maven project is analyzed.
2. Dependencies and plugins from pom.xml are identified.
3. Gradle configuration files are created.
4. Maven dependencies are converted into Gradle dependencies.
5. The project is rebuilt and tested using Gradle.

Advantages

- Faster build execution
- Flexible project configuration
- Better performance with incremental builds
- Easier dependency management
- Improved CI/CD integration

Applications

- Web application development
- Selenium automation projects
- Continuous Integration and Deployment pipelines
- DevOps automation workflows

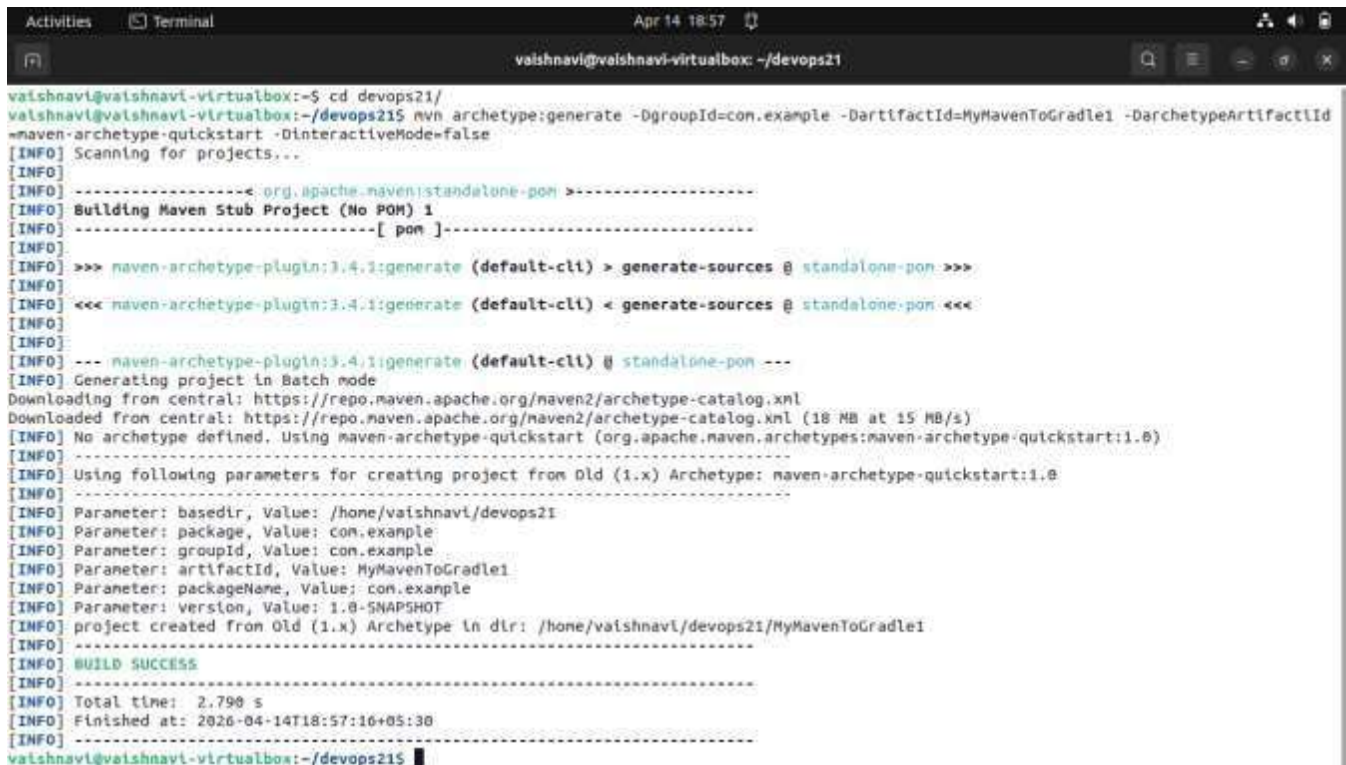
This conversion helps developers use Gradle's advanced build automation features while maintaining the existing project functionality.

Steps to execute the project:

Step 1:

Create a simple maven project with the command:

```
mvn archetype:generate -DgroupId=com.example -DartifactId=MyMavenToGradle -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
```



```
vaishnavi@vaishnavi-virtualbox:~/devops21$ mvn archetype:generate -DgroupId=com.example -DartifactId=MyMavenToGradle -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
[INFO] Scanning for projects...
[INFO] Building Maven Stub Project (No POM) 1
[INFO] [ pom ]
[INFO] >>> mvn archetype-plugin:3.4.1:generate (default-cli) > generate-sources @ standalone-pom >>>
[INFO] <<< mvn archetype-plugin:3.4.1:generate (default-cli) < generate-sources @ standalone-pom <<<
[INFO] --- mvn archetype-plugin:3.4.1:generate (default-cli) @ standalone-pom ---
[INFO] Generating project in Batch mode
[INFO] Downloading from central: https://repo.maven.apache.org/maven2/archetype-catalog.xml
[INFO] Downloaded from central: https://repo.maven.apache.org/maven2/archetype-catalog.xml (18 MB at 15 MB/s)
[INFO] No archetype defined. Using maven-archetype-quickstart (org.apache.maven.archetypes:maven-archetype-quickstart:1.0)
[INFO] Using following parameters for creating project from Old (1.x) Archetype: maven-archetype-quickstart:1.0
[INFO] Parameter: basedir, Value: /home/vaishnavi/devops21
[INFO] Parameter: package, Value: com.example
[INFO] Parameter: groupId, Value: com.example
[INFO] Parameter: artifactId, Value: MyMavenToGradle
[INFO] Parameter: packageName, Value: com.example
[INFO] Parameter: version, Value: 1.0-SNAPSHOT
[INFO] project created from Old (1.x) Archetype in dir: /home/vaishnavi/devops21/MyMavenToGradle
[INFO] BUILD SUCCESS
[INFO] Total time: 2.790 s
[INFO] Finished at: 2020-04-14T18:57:16+05:30
[INFO] vaishnavi@vaishnavi-virtualbox:~/devops21$
```

The command can be broken down as:

- DgroupId=com.example – project's base package (reverse-domain style).
- DartifactId=my-maven-app – defines the name of the project (will also be the folder name).
- DarchetypeArtifactId=maven-archetype-quickstart – sets the Maven template to be the basic Java application.
- DinteractiveMode=false – deactivates interactive mode and uses the given values automatically.

Step 2:

Edit the file pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/maven-v4_0_0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.example</groupId>
  <artifactId>MyMavenApp</artifactId>
  <packaging>jar</packaging>
```

```
<version>1.0-SNAPSHOT</version>
<name>MyMavenApp</name>

<url>http://maven.apache.org</url>

<!-- Properties: Customize Java version or plugin versions -->
<properties>
<maven.compiler.source>11</maven.compiler.source>
<maven.compiler.target>11</maven.compiler.target>
</properties>
<dependencies>
  <dependency>
    <groupId>junit</groupId>
    <artifactId>junit</artifactId>
    <version>3.8.1</version>
    <scope>test</scope>
  </dependency>
</dependencies>

<!-- Build: Configuring plugins and build settings -->
<build>
<plugins>
<!-- Example: Maven Compiler Plugin to compile Java code -->
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-compiler-plugin</artifactId>
<version>3.8.1</version>
<configuration>
<source>1.7</source>
<target>1.7</target>
</configuration>
</plugin>
<!-- Example: Maven Surefire Plugin to run tests -->
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-surefire-plugin</artifactId>
```

Build the project using the command :

```

Activities Terminal Apr 14 19:00
vaishnavi@vaishnavi-virtualbox: ~/devops21/MyMavenToGradle1
INFO --- maven-clean-plugin:2.5.1:clean (default-clean) @ MyMavenApp ---
INFO Deleting /home/vaishnavi/devops21/MyMavenToGradle1/target
INFO
INFO --- maven-resources-plugin:2.8:resources (default-resources) @ MyMavenApp ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
INFO skip non existing resourceDirectory /home/vaishnavi/devops21/MyMavenToGradle1/src/main/resources
INFO
INFO --- maven-compiler-plugin:3.8.1:compile (default-compile) @ MyMavenApp ---
INFO Changes detected - recompiling the module!
[WARNING] File encoding has not been set, using platform encoding UTF-8, i.e. build is platform dependent!
INFO Compiling 1 source file to /home/vaishnavi/devops21/MyMavenToGradle1/target/classes
INFO
INFO --- maven-resources-plugin:2.8:testResources (default-testResources) @ MyMavenApp ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
INFO skip non existing resourceDirectory /home/vaishnavi/devops21/MyMavenToGradle1/src/test/resources
INFO
INFO --- maven-compiler-plugin:3.8.1:testCompile (default-testCompile) @ MyMavenApp ---
INFO Changes detected - recompiling the module!
[WARNING] File encoding has not been set, using platform encoding UTF-8, i.e. build is platform dependent!
INFO Compiling 1 source file to /home/vaishnavi/devops21/MyMavenToGradle1/target/test-classes
INFO
INFO --- maven-surefire-plugin:2.22.1:test (default-test) @ MyMavenApp ---
INFO
INFO
INFO .....
INFO
INFO Running com.example.AppTest
INFO Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.003 s - in com.example.AppTest
INFO
INFO Results:
INFO
INFO Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
INFO
INFO
INFO --- maven-jar-plugin:3.5.0:jar (default-jar) @ MyMavenApp ---
INFO Building jar: /home/vaishnavi/devops21/MyMavenToGradle1/target/MyMavenApp-1.0-SNAPSHOT.jar
INFO
INFO --- maven-install-plugin:2.8:install (default-install) @ MyMavenApp ---
INFO Installing /home/vaishnavi/devops21/MyMavenToGradle1/target/MyMavenApp-1.0-SNAPSHOT.jar to /home/vaishnavi/.m2/repository/com/example/MyMavenApp/1.0-SNAPSHOT/MyMavenApp-1.0-SNAPSHOT.jar
INFO
INFO .....
INFO BUILD SUCCESS
INFO
INFO .....
INFO Total time: 5.374 s
INFO Finished at: 2025-04-14T19:05:05+05:30
INFO
vaishnavi@vaishnavi-virtualbox: ~/devops21/MyMavenToGradle1$

```

If the command shows output as “build success”, it means that the project has compiled successfully

Step 4:

Migrating the maven project to gradle. Execute the commands given below:

```
$git init --type pom.xml
```

```
[INFO] -----
vaishnavi@vaishnavi-virtualbox:~/devops21/MyMavenToGradle1$ gradle init --type pom.xml
Starting a Gradle Daemon, 1 incompatible and 1 stopped Daemons could not be reused, use --status for details

FAILURE: Build failed with an exception.

* What went wrong:
Execution failed for task ':init'.
> The requested build setup type 'pom.xml' is not supported. Supported types: 'basic', 'groovy-application', 'groovy-library', 'java-application', 'java-library', 'pom', 'scala-library'.

* Try:
Run with --stacktrace option to get the stack trace. Run with --info or --debug option to get more log output. Run with --scan to get full insights.

* Get more help at https://help.gradle.org

BUILD FAILED in 3s
2 actionable tasks: 2 executed
vaishnavi@vaishnavi-virtualbox:~/devops21/MyMavenToGradle1$
```

Error explanation:

- ❗ pom.xml is the name of a configuration file used in Maven projects, not a project type. Gradle does not recognize file names in the --type option.
- ❗ The --type flag in Gradle expects a valid project type keyword (like java-application, basic, or pom) that tells Gradle what kind of project to create.
- ❗ Since pom.xml is not in Gradle’s list of supported types, it throws an error saying it is not supported. On the other hand, pom is a valid type in Gradle, used when you want to create a project compatible with Maven-style structure.

Step 5:

Execute the command to view the files: \$ls

There is no build.gradle file in the project.

Step 6:

To correct the error, execute the command:

```
$git init --type pom
```

```
2 actionable tasks: 2 executed
vaishnavi@vaishnavi-virtualbox:~/devops21/MyMavenToGradle1$ ls
gradle gradlew gradlew.bat pom.xml src target
vaishnavi@vaishnavi-virtualbox:~/devops21/MyMavenToGradle1$ gradle init --type pom

> Task :init
Maven to Gradle conversion is an incubating feature.

BUILD SUCCESSFUL in 2s
2 actionable tasks: 1 executed, 1 up-to-date
vaishnavi@vaishnavi-virtualbox:~/devops21/MyMavenToGradle1$
```

gradle init --type pom is used to initialize (create) a new Gradle project

It tells Gradle to create the project in a Maven-style structure (similar to how Maven projects are organized) It generates

basic files like:

- build.gradle (Gradle build file)
- project folder structure (like src/main/java, etc.)

The pom type specifically is used when you want a simple project setup or a parent project (like Maven's pom packaging)

Step 6:

Execute the same command again or the given command to view the project structure.

\$tree

```

vaishnavi@vaishnavi-virtualbox: ~/devops21/MyMavenToGradle1$ tree
.
├── build.gradle
├── gradle
│   └── wrapper
│       ├── gradle-wrapper.jar
│       └── gradle-wrapper.properties
├── gradlew
├── gradlew.bat
├── pom.xml
├── settings.gradle
├── src
│   ├── main
│   │   ├── java
│   │   │   ├── com
│   │   │   │   └── example
│   │   │       └── App.java
│   └── test
│       ├── java
│       │   ├── com
│       │   │   └── example
│       │       └── AppTest.java
├── target
│   ├── classes
│   │   ├── com
│   │   │   └── example
│   │       └── App.class
│   ├── generated-sources
│   │   └── annotations
│   ├── generated-test-sources
│   │   └── test-annotations
│   ├── maven-archiver
│   │   └── pom.properties
│   ├── maven-status
│   │   └── maven-compiler-plugin
│   │       ├── compile
│   │       │   ├── default-compile
│   │       │   ├── createdFiles.lst
│   │       │   └── inputFiles.lst
│   │       └── testCompile
│   │           ├── default-testCompile
│   │           ├── createdFiles.lst
│   │           └── inputFiles.lst
│   └── MyMavenApp-1.0-SNAPSHOT.jar
├── surefire-reports
│   ├── con.example.AppTest.txt
│   └── TEST-con.example.AppTest.xml
└── test-classes
  
```

Now, we can make the given observation in the output: This time build.gradle file is created n included in the file

Pushing into the Github

git config --global user.name "vaishnavim1121"

\$ git config --global user.email "vaishnavim1121@gmail.com"

\$git init \$git add . \$git commit -m "initial commit"

\$ git remote add origin <https://github.com/vaishnavim1121/MyMavenToGradle.git>

\$ git remote set-url origin https://*PAT*@github.com/vaishnavim1121/MyMavenToGradle

\$ git push -u origin master


```

Activities Terminal Apr 14 19:10
vaishnavi@vaishnavi-virtualbox: ~/devops21/MyMavenToGradle1
AppTest.class

30 #directories, 19 files
vaishnavi@vaishnavi-virtualbox:~/devops21/MyMavenToGradle1$ git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint: git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint: git branch -m <name>
Initialized empty Git repository in /home/vaishnavi/devops21/MyMavenToGradle1/.git/
vaishnavi@vaishnavi-virtualbox:~/devops21/MyMavenToGradle1$ git add .
vaishnavi@vaishnavi-virtualbox:~/devops21/MyMavenToGradle1$ git commit -m "maven to gradle"
[master (root-commit) 255ca2d] maven to gradle
27 files changed, 473 insertions(+)
create mode 100644 .gradle/4.4.1/fileChanges/last-build.bin
create mode 100644 .gradle/4.4.1/fileHashes/fileHashes.bin
create mode 100644 .gradle/4.4.1/fileHashes/fileHashes.lock
create mode 100644 .gradle/4.4.1/taskHistory/taskHistory.bin
create mode 100644 .gradle/4.4.1/taskHistory/taskHistory.lock
create mode 100644 .gradle/buildOutputCleanup/buildOutputCleanup.lock
create mode 100644 .gradle/buildOutputCleanup/cache.properties
create mode 100644 .gradle/buildOutputCleanup/outputFiles.bin
create mode 100644 build.gradle
create mode 100644 gradle/wrapper/gradle-wrapper.jar
create mode 100644 gradle/wrapper/gradle-wrapper.properties
create mode 100755 gradlew
create mode 100644 gradlew.bat
create mode 100644 pom.xml
create mode 100644 settings.gradle
create mode 100644 src/main/java/com/example/App.java
create mode 100644 src/test/java/com/example/AppTest.java
create mode 100644 target/MavenApp-1.0-SNAPSHOT.jar
create mode 100644 target/classes/com/example/App.class
create mode 100644 target/maven-archiver/pom.properties
create mode 100644 target/maven-status/maven-compiler-plugin/compile/default-compile/createdFiles.lst
create mode 100644 target/maven-status/maven-compiler-plugin/compile/default-compile/inputFiles.lst
create mode 100644 target/maven-status/maven-compiler-plugin/testCompile/default-testCompile/createdFiles.lst
create mode 100644 target/maven-status/maven-compiler-plugin/testCompile/default-testCompile/inputFiles.lst
create mode 100644 target/surefire-reports/TEST-com.example.AppTest.xml
create mode 100644 target/surefire-reports/com.example.AppTest.txt
create mode 100644 target/test-classes/com/example/AppTest.class
vaishnavi@vaishnavi-virtualbox:~/devops21/MyMavenToGradle1$ git remote add origin https://github.com/vaishnavini121/MyMavenToGradle.git
git: 'remote' is not a git command. See 'git --help'.

remote
vaishnavi@vaishnavi-virtualbox:~/devops21/MyMavenToGradle1$ git remote add origin https://github.com/vaishnavini121/MyMavenToGradle.git
vaishnavi@vaishnavi-virtualbox:~/devops21/MyMavenToGradle1$ git remote set-url origin https://ghp_NmHyPMp5Sxgt2QwFOV7noGjtJaL6ZV3wIFI@github.com/vaishnavini121/MyMavenToGradle
vaishnavi@vaishnavi-virtualbox:~/devops21/MyMavenToGradle1$ git push -u origin master

vaishnavi@vaishnavi-virtualbox:~/devops21/MyMavenToGradle1$ git push -f origin master
Enumerating objects: 61, done.
Counting objects: 100% (61/61), done.
Delta compression using up to 10 threads
Compressing objects: 100% (34/34), done.
Writing objects: 100% (61/61), 61.01 KiB | 8.72 MiB/s, done.
Total 61 (delta 2), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (2/2), done.
To https://github.com/vaishnavini121/MyMavenToGradle
+ 17f5a91...255ca2d master -> master (forced update)
vaishnavi@vaishnavi-virtualbox:~/devops21/MyMavenToGradle1$
vaishnavi@vaishnavi-virtualbox:~/devops21/MyMavenToGradle1$

```

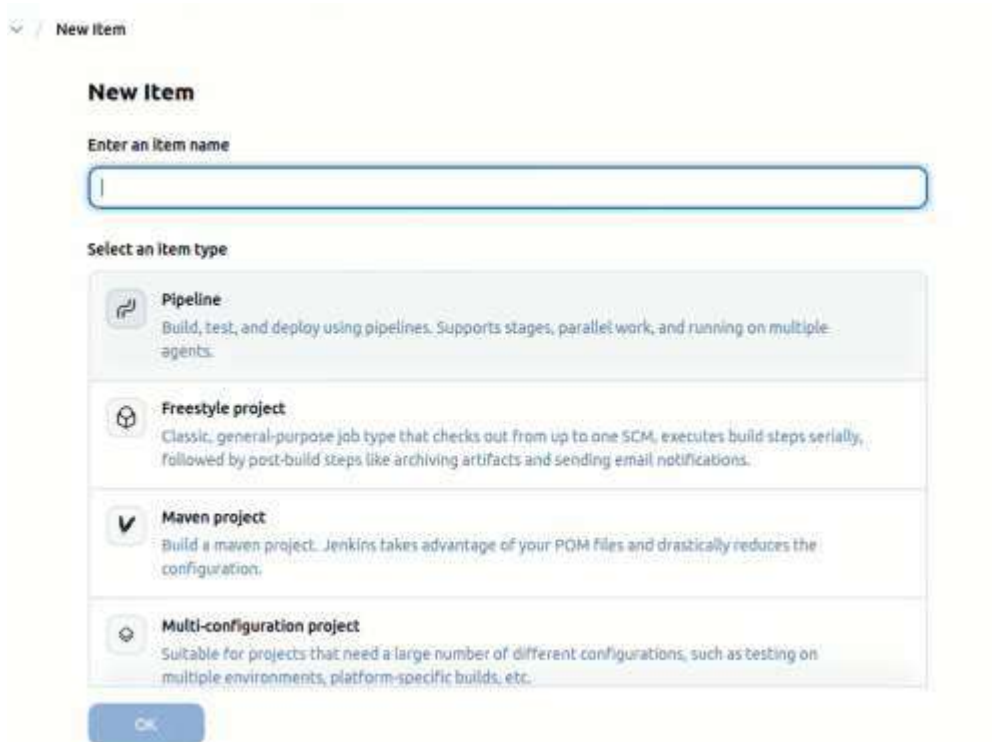
Second Phase

Depicting Jenkins Pipeline

Follow the given steps to deploy the project on Jenkins.

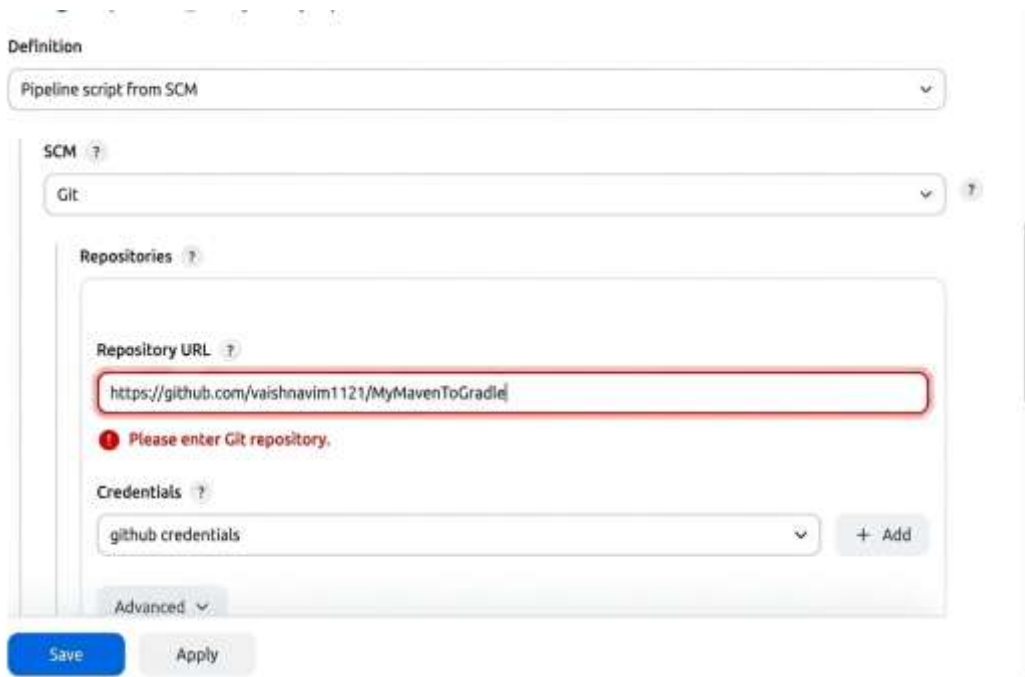
create a Jenkins pipeline job that will use the Jenkinsfile from your project repository

go to the Jenkins home page and click on “New Item” to create a job, enter the item name select Pipeline as the project type.



The image shows the 'New Item' configuration page in Jenkins. At the top, there is a 'New Item' header and a 'Back' button. Below this is a text input field labeled 'Enter an item name'. Underneath is a section titled 'Select an item type' with four options: 'Pipeline' (with a Jenkins logo icon), 'Freestyle project' (with a gear icon), 'Maven project' (with a checkmark icon), and 'Multi-configuration project' (with a gear icon). Each option has a brief description. The 'Maven project' option is currently selected. At the bottom of this section is an 'OK' button.

This option allows you to build, test, and deploy applications using a Jenkinsfile. After selecting the pipeline type, click “OK” to proceed to the job configuration page



The image shows the 'Pipeline' configuration page in Jenkins. The 'Definition' dropdown is set to 'Pipeline script from SCM'. The 'SCM' dropdown is set to 'Git'. Below this is the 'Repositories' section, which contains a 'Repository URL' field with the value 'https://github.com/vaishnavim1121/MyMavenToGradle'. A red error message 'Please enter Git repository.' is displayed below the URL field. The 'Credentials' dropdown is set to 'github credentials', and there is an '+ Add' button next to it. At the bottom of the configuration area is an 'Advanced' dropdown. At the very bottom are 'Save' and 'Apply' buttons.

Now we need to add a jenkinsfile into our project which will consist of the operations that will occur when building the project in Jenkins

It includes step wise instructions guiding at each stage with right commands


```

1 pipeline {
2   agent any // Use any available agent
3
4   tools {
5     gradle 'gradle' // Ensure this matches the name configured in Jenkins
6     jdk 'jdk'
7   }
8   stages {
9     stage('Checkout') {
10      steps {
11        git branch: 'master', url: 'https://github.com/vaishnavi1121/MyGradleApp.git'
12      }
13    }
14
15    stage('Build') {
16      steps {
17        sh 'gradle build' // Run Gradle build
18      }
19    }
20
21    stage('Test') {
22      steps {
23        sh 'gradle test' // Run unit tests
24      }
25    }
26
27    stage('Run Application') {
28      steps {
29        // Start the JAR application
30        sh 'gradle run'
31      }
32    }
33  }
34 }

```

Now try building the project with Jenkins

The screenshot shows the Jenkins web interface for a pipeline named 'MyMavenToGradle'. The pipeline is in a 'Success' state, indicated by a green checkmark. The 'Stage View' section displays a table of stage execution times for the current build (11) and the average stage times.

Stage	Declarative: Checkout SCM	Declarative: Tool Install	Checkout	Build	Test	Run Application	Declarative: Post Actions
Average stage times: (Full run time: ~33s)	1s	448ms	3s	18s	3s	3s	231ms
11: 11:38	1s	448ms	3s	18s	3s	3s	231ms

The 'Permalinks' section provides links to various build types:

- Last build (#1), 26 days ago
- Last stable build (#1), 26 days ago
- Last successful build (#1), 26 days ago
- Last completed build (#1), 26 days ago

The bottom of the interface shows the 'Builds' section with a search bar and a list of builds. The current build (11) is shown with a green status icon and a duration of 11:38. The footer indicates the REST API and Jenkins version 2.562.

The project is successfully built